Face Recognition

V2.1. 0

API Documentation

Version and Modification Records

Version	Description	Date
V1.0	Create	2019-03-29
V1.1	Added Living Mode Settings mipsSetLivenessMode mipsinit The function of Monocular living body recognition is added Add required model files Modify Face Motion Detection Default value addOneFaceToDB , more conditional judgements are added. Add mipsVerifyInFaceDB Add mipsGetFeature mipsFaceInfoTrack add more face attributes	2019-06-06
V1.2	1. mipsinit add the distance and angle parameters 2. mipsFaceInfoTrackLiving Class renamed to mipsFaceInfoTrack 3. mipsFaceTrackLiving Class renamed to mipsVideoFaceTrack 4.Increase mipsGetAlgorithmType	2019-06-13
V2.0	1.add verifyVipImage Obtain detailed Face information of pictures 2.add mipsCheckVipIdValid check Vipid validation	2019-08-05
V2.0. 1	1.add Face Recognition Region related API Only in the settings area can do the face identification and live detection. mipsEnableFaceVerifyArea mipsDisableFaceVerifyArea mipsGetFaceVerifyAreaState mipsSetFaceVerifyArea mipsGetFaceVerifyArea	2019-09-29
V2.0. 2	1.add VIP Detection times related API , only VIP identification turned on then these APIs can take effect: mipsSetMaxVipVerifyCntThrehold mipsGetVipVerifyCntThrehold 2.Fix the bug of common version using features to add face database.	2019-10-24
V2.0. 3	1. Add initialization callback interface: mipsInit 2. Add clear flag interface: mipsResetTracker	2020-01-14

V2.0. 5	Add support for face attributes (mask Detection): **mipsEnableFaceAttr** **mipsDisableFaceAttr** **mipsGetFaceAttrState** **mipsEnableRefreshFaceAttr** **mipssDisableRefreshFaceAttr** **mipsGetRefreshFaceAttrState** Add support for wearing mask mode: **mipsSetSimilarityThreholdMask** **mipsGetSimilarityThreholdMask** **mipsSetFaceScoreThreholdVIP** **mipsGetFaceScoreThreholdVIP**	2020-02-26
V2.1. 0	1, compatible with Sensetime Algorithm V3.5. 2, add interfaces mipsGetAlgorithmInfo	2020-03-22

I. Module Introduction

This module provides a basic module for Android applications, which is suitable for application scenarios such as access comparison equipment and face recognition gate. Its purpose is to read the picture of the camera and detect the face in the picture. SDK supports single camera and can perform the following operations at the same time:

Face database search (1: N comparison): Firstly, a face database (less than or equal 20,000 people) is registered with the face image of person, and the face feature data can be stored. Then, in the process of real-time detection, the features of the face in the camera are extracted and compared with the features in the face database (the highlighting similarity is generated), and where the current face is in the face database is identified according to the similarity. To ensure the accuracy of search and comparison, the face registered into the face database needs to ensure the whole face, and the width of the face frame is greater than 90 pixels.

Face comparison (1: 1 comparison): Mainly used for simple scenes such as face+card comparison. Comparing with the photo and the camera picture (or the photo) are the same person: extracting the face features in the photo and the camera picture, and then comparing the two face features to check similarity, which is greater than the threshold value, then it can be considered as the same person;

Note: This module current includes 1: 1 comparison and 1:N face search.

The SDK contains android SDK: MIPS_AI_Basic_ BinocularIDCheckOffline.jar (support single and dual-camera devices). Import the Jar file and copy the files under *model* folder into assets folder. The files in jniLibs should also copy to the corresponding folder in your project.

II. Face SDK API

2.1 API List

Type	API name	remark
Get info	mipsGetVersion	Get SDK version
	mipsGetAlgorithmType	Get type of Algorithm
	mipsGetAlgorithmInfo	Get current algorithm info
Initialization	mipsInit	Initialize the SDK
Un-init	mipsUninit	
Parameter setter and getter	mipsSetRollAngle mipsGetRollAngle	Set/Get When detecting, check the roll angle value of the face (set range 1-89, 5-45 recommended)
	mipsSetYawAngle mipsGetYawAngle	Set/Get When detecting, check the Yaw angle value of the face (set range 1-89, 5-45 recommended)
	mipsSetPitchAngle mipsGetPitchAngle	Set/Get When detecting, check the Pitch angle value of the face (set range 1-89, 5-45 recommended)
	mipsSetFaceScoreThrehold mipsGetFaceScoreThrehold	Set/get When detecting, Face confidence threshold (set range 0.1-1.0, use default value)
	mipsSetVerifyScoreThrehold mipsGetVerifyScoreThrehold	Set/Get Face confidence threshold of face image (set range 0.1-1.0, use default value)
	mipsSetSimilarityThrehold mipsGetSimilarityThrehold	Set/Get threshold value of face similarity (set range 0.1-1.0, recommend 0.5-0.8)
	mipsSetMaxFaceTrackCnt mipsGetMaxFaceTrackCnt	Set/Get max face detect number in the single frame (set range 1-32, recommend 4)
	mipsSetFaceWidthThrehold mipsGetFaceWidthThrehold	Set/Get When detecting face, the min width of face (set range 1-min width of input frame, recommend 30-200)
	mipsSetPicFaceWidthThrehold mipsGetPicFaceWidthThrehold	Set/Get the min width of face in picture (set range 1-min width of input frame, recommend 30-200)
	mipsSetPicRollAngle mipsGetPicRollAngle	Set/Get roll angle of face detecting in image. (set range 1-89, recommend 5-15, default 10)
	mipsSetPicYawAngle mipsGetPicYawAngle	Set/Get Yaw angle of face detecting in image. (set range 1-89, recommend 5-45, default 10)
	mipsSetPicPitchAngle mipsGetPicPitchAngle	Set/Get Pitch angle of face detecting in image. (set range 1-89, recommend 5-45, default 10)
	mipsSetVIPDetectFaceWidthThrehold mipsGetVIPDetectFaceWidthThrehold	Set/Get When detecting registered user face, the min width of face (set range 1-min width of input frame, recommend 30-200)
	mipsSetLivenessFaceWidthThrehold mipsGetLivenessFaceWidthThrehold	Set/get face width value in liveness detection mode. (set range 1-min of width and height, recommend camera input mode)
	mipsSetTrackLandscape mipsSetTrackPortrait mipsSetTrackReverseLandscape mipsSetTrackReversePortrait	Set different direction of camera stream input.
	mipsSetVIPRollAngle mipsGetVIPRollAngle	Set/Get the roll angle of registered user in face database (set range 1-89,recommend 5-45, default 20)
	mipsSetVIPYawAngle mipsGetVIPYawAngle	Set/Get Yaw angle of registered user in face database (set range 1-89,recommend 5-45, default 25)

	mipsSetVIPPitchAngle mipsGetVIPPitchAngle	Set/Get Pitch angle of registered user in face database (set range 1-89,recommend 5-45, default 20)
	mipsSetIDProCropFaceBorderwidth mipsGetIDProCropFaceBorderwidth	Set/get the border width of face image to extract the characteristics
	mipsSetLivenessMode mipsGetLivenessMode	Set/get liveness detection mode
	mipsSetSimilarityThreholdMask mipsGetSimilarityThreholdMask	Set/Get threshold value of face similarity under the mask
	mipsSetFaceScoreThreholdVIP mipsGetFaceScoreThreholdVIP	Set/Get threshold value of face match
	mipsEnableFaceAttr mipsDisableFaceAttr mipsGetFaceAttrState	Get the attribute value of face detected in Realtime identifying (default enabled)
	mipsEnableRefreshFaceAttr mipssDisableRefreshFaceAttr mipsGetRefreshFaceAttrState	Refresh the face attribute value(default disabled)
Real-time identification	mipsDetectOneFrame	Video stream detection frame by frame. Can get face attributes Recommend use this interface
	mipsGetFaceCnt	Get number of face detected
	mipsGetDbFaceCnt	Get the total number of registered face
	mipsGetFaceInfoDetected	Get the user info that identified
	mipsEnableVipFaceVerify mipsDisableVipFaceVerify mipsGetVipFaceVerifyState	VIP identification during the real-time detection. (default disabled)
	mipsEnableIDFeature mipsDisableIDFeature mipsGetIDFeatureState	Get the face feature data (default disabled)
	mipsEnableFaceLiveness mipsDisableFaceLiveness mipsGetFaceLivenessState	Liveness detection mode (default enabled, Need to set the last parameter of *mipsInit* interface, then it will work)
	mipsEnableRefreshFaceLiveness mipsDisableRefreshFaceLiveness mipsGetRefreshFaceLivenessState	Refresh the liveness detection status (default disabled)
	mipsEnableRefreshFaceVIP mipsDisableRefreshFaceVIP mipsGetRefreshFaceVIPState	Refresh the VIP face check status (default disabled)
	mipsEnableRefreshFaceRect mipsDisableRefreshFaceRect	Refresh the location of face detection frame in each frame of stream.
	mipsEnableIDFeatureCropFace mipsDisableIDFeatureCropFace mipsGetIDFeatureCropFaceState	Crop face image when extracting the features (default disabled)
	Function switch interface: mipsEnableFaceMoveDetect 、mipsDisableFaceMoveDetect、 mipsGetFaceMoveDetectState detect threshold value setting: mipsSetFaceMoveRitio_THRESHOLD、 mipsGetFaceMoveRitio_THRESHOLD Mode of detection: mipsSetFaceMoveDetectRatioMode、 mipsGetFaceMoveDetectRatioMode time of detection mipsSetFaceMoveDetectTimer、mipsGetFaceMoveDetectTimer	Dynamic detection mode

	mipsEnableFaceVerifyArea **mipsDisableFaceVerifyArea** **mipsGetFaceVerifyAreaState**	**Face detection area setting (default disabled)**
	mipsSetFaceVerifyArea **mipsGetFaceVerifyArea**	**Set/Get identify area setting（activate by enable the face verify area function）**
	mipsSetMaxVipVerifyCntThrehold **mipsGetVipVerifyCntThrehold**	**Set/Get max time of VIP verified (only activate when VIP refresh function enabled)**
Face database	**initFaceDB**	**Initialize the face Database**
	saveFaceDB	**Save face DB**
	mipsCheckVipIdValid	**vipId validation check**
	addOneFaceToDB	**Add one face into face DB**
	deleteOneFaceFrDB	**Delete one face from DB**
	deleteAllFaceFrDB	**Delete all face data**
	verifyVipImage	**Check image quality can be used for face enroll**
	mipsGetFeature	**Extract the face feature data in image**
	mipsGetFeatureFrDB	**Get face feature with specific ID**
	deleteBulkFacesFrDB	**Bulk delete the face data**
Face match	**mipsVerifyID**	**1:1 match face feature**
Face search	**mipsVerifyInFaceDB**	**1:N match, search face ID which matched with camera captured**

2.2 mipsVideoFaceTrack API Definition

2.2.1 Basic Information Interface

- **static String mipsGetVersion()**

Definition：Get SDK version number.

API table：

Parameters/return value	Type	Description	Example
Return value	String	SDK version number	

Example code：

String ver= mipsVideoFaceTrack.mipsGetVersion();

- **String mipsGetAlgorithmType(Context context)**

Definition：get the type of algorithm.

API table：

Parameters/return value	Type	Description	Example
Return value	String	Type of algorithm: SenseTime default	

Example code：

String type=mipsVideoFaceTrack.mipsGetAlgorithmType(context));

- **String mipsGetAlgorithmInfo()**

Definition：Get information of algorithm

API table：

Parameters/return value	Type	Description	Example
Return value	String	Type of algorithm: "SenseTime IDCheck V1" "SenseTime IDCheck V2" "SenseTime IDCheck V3.5.4" (in used) "SenseTime Basic V1" "SenseTime Insight V1" "ReadFace V1"	

Notice：all the algorithm list above needs the specific encrypted IC on the main board then can be used.

Example code：

String type=mipsVideoFaceTrack. mipsGetAlgorithmInfo (context));

2.2.2 Initialization

- ***int mipsInit(Context context, int distanceType, int degree, String pathMipsLicFile, int flgLiveness)***

Definition：Module initialization

API table：

Parameters/return value	**Type**	**Description**	**Example**
context	Context		
distanceType	int	Face detect distance： 0: <1m , 1: 1~2m , 2: 2~4m , 3: 4~6m	
degree	int	Camera angle setting, can set 0, 90, 180, 270	
pathMipsLicFile	String	Path of license file	/sdcard/mipsLic/mipsAi.lic
flgLiveness	int	2：Enable dual camera detection 1：Enable single camera detection 0：Disable liveness detection	
Return value	int	0: success -1：hardware license failed -2：Face tracking failed or not support single camera liveness 3：Liveness initialization failed -5：Face match initialization failed -6：Face detect initialization failed -7：License file error	

Example code：

int ret=mfaceDetect.mipsInit(context, 2, 90 , "/sdcard/mipsLic/mipsAi.lic", 2);

- ***void mipsInit(Context context, int distanceType, int degree, String pathMipsLicFile, int flgLiveness, InitCallback initCallback)***

Definition：module initialization

API table：

Parameters/return value	**Type**	**Description**	**Example**
context	Context		
distanceType	int	Face detect distance： 0: <1m , 1: 1~2m , 2: 2~4m , 3: 4~6m	
degree	int	Camera angle setting, can set 0, 90, 180, 270	
pathMipsLicFile	String	Path of license file	/sdcard/mipsLic/mipsAi.lic
flgLiveness	int	2：Enable dual camera detection 1：Enable single camera detection 0：Disable liveness detection	
initCallback	InitCallback	Callback method: Success execute *onSucceed()*, Failed execute *onFailed(int returnCode)*, *returnCode:* 0: success -1：hardware license failed -2：Face tracking failed or not support single camera liveness 3：Liveness initialization failed -5：Face match initialization failed -6：Face detect initialization failed -7：License file error	

Example code：

int ret=mfaceDetect.mipsInit(context, 2, 90 , "/sdcard/mipsLic/mipsAi.lic", 5, initCallback)

- *void mipsUninit()*

Definition：module un-initialization

2.2.3 Real-time identification

int mipsDetectOneFrame(byte[] frameImage, int frame_width, int frame_height ,int trackIDLiveness)

Definition：identify the face information from single camera.

API table：

Parameters/return value	**Type**	**Description**	**Example**
frameImage	byte[]	Get one frame data from camera in NV21 format	
frame_width	int	Width of frame (pixel)	
frame_height	int	Height of frame (pixel)	
trackIDLiveness	int	TrackID which need liveness detect <0：no liveness detect	
Return value	int	0: face info no differences compare to the last image 1：face info is different from the last image -1：Check failed	

Notice：

This interface need to be used with *mipsGetFaceInfoDetected()*. When the return value is 1, need to use *mipsGetFaceInfoDetected()* to get the face info.

Three different situations with different return value:

Functions	**Method called**	**Return value as 1**
Liveness detection completed	---	Number of face changed
Liveness detect + VIP verify	*mipsEnableVipFaceVerify()*	Number of face changed Detected face, VIP verify not complete（flgSetVIP=0）
Liveness detect +dynamic refresh face detected rectangular frame	*mipsEnableRefreshFaceRect()*	Number of face changed Face detected in the area

Example code：

```
lockFaceDb.lock();
 flgFaceChange = mfaceDetect.mipsDetectOneFrame(tmp,PREVIEW_WIDTH,PREVIEW_HEIGHT，trackID);
 lockFaceDb.unlock();
 if(flgFaceChange == 1)
  {
    mcntCurFace = mfaceDetect.mipsGetFaceCnt();


    mdbFaceCnt = mfaceDetect.mipsGetDbFaceCnt();


    mFaceInfoDetected = mfaceDetect.mipsGetFaceInfoDetected();
    …
}
```

- *int mipsDetectOneFrame(byte[] frameImage, int frame_width, int frame_height, byte[] frameImageIR, int widthIR, int heightIR,int trackIDLiveness)*

Definition：identify the face information from dual-camera device

API table：

Parameters/return value	**Type**	**Description**	**Example**
frameImage	byte[]	Get one frame data from camera in NV21 format From RGB camera	
frame_width	int	Width of frame (pixel) From RGB camera	
frame_height	int	Height of frame (pixel) From RGB camera	
frameImageIR	byte[]	Get one frame data from camera in NV21 format From IR camera	
frame_widthIR	int	Width of frame (pixel) From IR camera	
frame_heightIR	int	Height of frame (pixel) From IR camera	
trackIDLiveness	int	TrackID which need liveness detect <0：no liveness detect	
Return value	int	0: face info no differences compare to the last image 1：face info is different from the last image -1：Check failed	

Notice：Hardware requires support dual-camera（RGB+IR）

- *int mipsGetFaceCnt()*

Definition：Get number of face detected

API table：

Parameters/return value	**Type**	**Description**	**Example**
Return value	int	Get number of face detected	

Notice:

Example code:

int mcntCurFace = mfaceDetect.mipsGetFaceCnt();

- *int mipsGetDbFaceCnt()*

Definition：Get the total number of registered face

API table：

Parameters/return value	**Type**	**Description**	**Example**
Return value	Int	the total number of registered face	

Example code:

int cnt= mfaceDetect.mipsGetDbFaceCnt()

- *mipsFaceInfoTrack [] mipsGetFaceInfoDetected()*

Definition：Get the user info that identified
API table：

Parameters/return value	Type	Description	Example
Return value	mipsFaceInfoTrack []	Get the user info that identified	

Notice：This interface should be used with *mipsDetectOneFrame()*

Example code：
mipsFaceInfoTrack [] mFaceInfoDetected = mfaceDetect.mipsGetFaceInfoDetected();

- *void mipsSetTrackLandscape( )*

Definition：Set camera video stream in landscape direction（Default setting）

- *void mipsSetTrackPortrait( )*

Definition：Set camera video stream in portrait direction

- *void mipsSetTrackReverseLandscape( )*

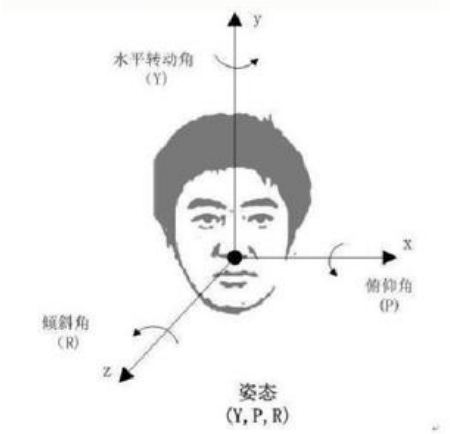
Definition：Set camera video stream in reversed landscape direction

- *void mipsSetTrackReversePortrait( )*

Definition：Set camera video stream in reversed portrait direction

- *void mipsSetRollAngle(float threhold)/ public float mipsGetRollAngle()*

Definition：Set/Get the roll angle of the face detected（if the angle larger than the setting value, it will not detect the face）



- *void mipsSetYawAngle(float threhold)/public float mipsGetYawAngle()*

Definition：Set/Get the Yaw angle of the face detected（if the angle larger than the setting value, it will not detect the face）

API table：

Parameters/return value	**Type**	**Description**	**Example**
Return value/ threhold	float	check the Yaw angle value of the face (set range 1-89, 5-45 recommended)	

- *void mipsSetPitchAngle(float threhold)/public float mipsGetPitchAngle()*

Definition：Set/Get the Pitch angle of the face detected（if the angle larger than the setting value, it will not detect the face）

API table：

Parameters/return value	**Type**	**Description**	**Example**
Return value/ threhold	float	check the Pitch angle value of the face (set range 1-89, 5-30 recommended)	

- *void mipsSetFaceScoreThrehold (float threhold)/public float mipsGetFaceScoreThrehold ()*

Definition：Set/Get Face confidence threshold（larger than the value then it will be detected as face）

API table：

Parameters/return value	**Type**	**Description**	**Example**
Return value/ threhold	float	Face confidence threshold (set range 0.1-1.0, use default value or >=0.5)	

- *void mipsSetVerifyScoreThrehold (float threhold)/public float mipsGetVerifyScoreThrehold ()*

Definition：Set/Get Face confidence threshold of face image

API table：

Parameters/return value	**Type**	**Description**	**Example**
Return value/ threhold	float	Face confidence threshold of face image（image quality）（use default value or >0.7）	

Notice：this interface will influence the following interfaces: *addOneFaceToDB(Context context ,mipsFaceVipDB vipFace), verifyVipImage(String[] imagePath, int imageCnt), mipsGetFeature(Bitmap bitmap), mipsFaceVipDB.*

Example code：

- *void mipsSetFaceScoreThreholdVIP (float threhold)/public float mipsGetFaceScoreThreholdVIP ()*

Definition：Set/Get threshold value of face match

API table：

Parameters/return value	**Type**	**Description**	**Example**
Return value/ threshold	float	threshold value of face match（use default value or >0.7）	

Notice：this interface will influence the following interfaces: *addOneFaceToDB(Context context ,mipsFaceVipDB vipFace), verifyVipImage(String[] imagePath, int imageCnt), mipsGetFeature(Bitmap bitmap), mipsFaceVipDB.*

Example code：

- *void mipsSetSimilarityThrehold (float threhold)/public float mipsGetSimilarityThrehold ()*

Definition：Set/Get threshold value of face similarity（larger than the threshold value will treat the person is the same in DB）

API table：

Parameters/return value	**Type**	**Description**	**Example**
Return value/ threshold	float	threshold value of face similarity (set range 0.1-1.0, recommend 0.5-0.8)	

- *void mipsSetSimilarityThreholdMask (float threhold)/public float mipsGetSimilarityThreholdMask ()*

Definition：Set/Get threshold value of face similarity under the mask（larger than the threshold value will treat the person is the same in DB）

API table：

Parameters/return value	**Type**	**Description**	**Example**
Return value/ threhold	float	threshold value of face similarity under the mask (set range 0.1-1.0, recommend 0.5-0.8)	

- *void mipsDisableFaceAttr ()*

Definition：disable function of getting the attribute value of face detected in Realtime identifying (default enabled)

- *int mipsGetFaceAttrState ()*

Definition：getting the attribute value of face detected (default enabled)
API table：

Parameters/return value	**Type**	**Description**	**Example**
Return value	int	0: disable； 1：enable	

- *void mipsEnableRefreshFaceAttr ()*

Definition：enable the real-time refresh the face attributes function (default disabled). Enable the function will use more system resource of the device, so make sure the function is needed.

- *void mipssDisableRefreshFaceAttr ()*

Definition：Disable the real-time refresh the face attributes function

- *int mipsGetRefreshFaceAttrState ()*

Definition：get current status of the real-time refresh the face attributes function

API table：

Parameters/return value	**Type**	**Description**	**Example**
Return value	int	0: disable 1：enable	

- *void mipsSetMaxFaceTrackCnt (int cnt)/public int mipsGetMaxFaceTrackCnt ()*

Definition：max face detect number in the single frame

API table：

Parameters/return value	**Type**	**Description**	**Example**
Return value	int	max face detect number in the single frame (>=4)	

Notice：the larger value set the more time cost for detection.

- *void mipsEnableVipFaceVerify()*

Definition：enable the 1:N identification（default disabled）. When in mipsFaceInfoTrack the flgSetVIP is 1, it means the ID (FaceIdxDB) is the value of identified person.

- *void mipsDisableVipFaceVerify()*

Definition：disable the 1:N identification

- *int mipsGetVipFaceVerifyState()*

Definition：get the status of 1:N identification function

API table：

Parameters/return value	**Type**	**Description**	**Example**
Return value	int	0: Disable 1: Enable	

- *void mipsEnableIDFeature()*

Definition：enable the 1:1 identification. When the *flgSetIDPro* is 1 in *mipsFaceInfoTracks* the mfaceFeature is valid value.

- *void mipsDisableIDFeature()*

Definition：Disable the 1:1 identification function.

- *int mipsGetIDFeatureState()*

Definition：get the status of 1:1 identification face feature extraction function

API table：

Parameters/return value	Type	Description	Example
Return value	int	0: Disable 1: Enable	

- *void mipsEnableRefreshFaceVIP()*

Definition：Enables real-time refresh of VIP verification function (off by default).When this function is enabled, the VIP check status will continue to be refreshed if it is not found in the face library, and the refresh will stop after the face library is found.The system overhead will increase after opening, please use with caution.

- *void mipsDisableRefreshFaceVIP()*

Definition：Disable real-time refresh of VIP check function (off by default)

- *int mipsGetRefreshFaceVIPState()*

Definition：Get whether the real-time refresh VIP check status function is on (off by default)

API table：

Parameters/return value	Type	Description	Example
Return value	int	0: Disable 1: Enable	

- *void mipsEnableRefreshFaceRect()*

Definition：Enables the refresh of the face rectangle. After calling this function, the detected face will also refresh the face rectangle in real time

- *void mipsDisableRefreshFaceRect()*

Definition：Disables the refresh of the face rectangle.

- *void mipsSetFaceWidthThrehold(int width)*

Definition：Set up track and witness than camera face feature extraction of the face width of box the smallest pixels (can be set range: 1 ~ (high input screen width minimum), the recommended 30 ~ 200) threshold, only the frame face width of box pixels reached the threshold will detect the human face (defaults to: 90), the threshold setting track detection face the greater the distance will be shorter.

API table：

Parameters/return value	Type	Description	Example
width	int	The minimum pixel width of a face frame	1. Suppose the threshold value of 90 pixels is set The distance between the face and the camera is 1.7 meters 2. Suppose a threshold value of 120 pixels is set The distance between the face and the camera is 1.3 meters

Notice：The lens focal length of different types of cameras is different, and the size of the face detected at the same distance and the same face will also be different. Generally, when the minimum pixel threshold of face frame width is set as 90, the distance between the face and the camera will be about 1.5 meters.

- *int mipsGetFaceWidthThrehold()*

Definition：Get the minimum pixel of face frame width of tracking and witness comparison camera to extract face features.

API table：

Parameters/return value	Type	Description	Example
Return value	Int	The minimum pixel width of a face frame	

- *void mipsSetPicFaceWidthThrehold (int width)*

Definition：Set the minimum pixel of image face detection and feature extraction face frame width [default: 90].

API table：

Parameters/return value	Type	Description	Example
width	int	The minimum pixel width of a face frame	

Notice：This interface set parameters affect the add faces library (addOneFaceToDB (Context Context, mipsFaceVipDB vipFace)), photographs, check (verifyVipImage (String [] imagePath, int imageCnt)), extract features from images (mipsGetFeature (Bitmap Bitmap)) interface, such as mipsFaceVipDB, mipsFaceVipDB

- *int mipsGetPicFaceWidthThrehold ()*

Definition：Get the image face detection, feature extraction face frame width of the minimum pixel.

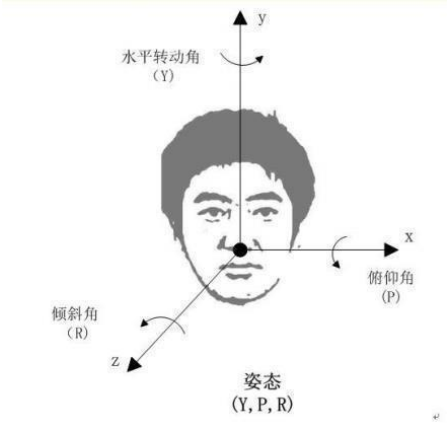
- *void mipsSetPicRollAngle(float threhold)/ public float mipsGetPicRollAngle()*

Definition：Set/Get Image detection face roll angle threshold (greater than this Angle will not be reported as a face)

API table：

Parameters/return value	Type	Description	Example
Return value/ threhold	float	Detection of face roll angle threshold (suggested setting range :<15.0)	

Notice：This interface set parameters affect the add faces library (addOneFaceToDB (Context Context, mipsFaceVipDB vipFace)), photographs, check (verifyVipImage (String [] imagePath, int imageCnt)), extract features from images (mipsGetFeature (Bitmap Bitmap)) interface, such as mipsFaceVipDB, mipsFaceVipDB



- *void mipsSetPicYawAngle(float threhold)/public float mipsGetPicYawAngle()*

Definition：Set/Get Image detection face yaw angle threshold (greater than this Angle will not be reported as a face)

API table：

Parameters/return value	Type	Description	Example
Return value/ threhold	float	Detection of face yaw angle threshold (suggested setting range :<15.0)	

Notice：This interface set parameters affect the add faces library (addOneFaceToDB (Context Context, mipsFaceVipDB vipFace)), photographs, check (verifyVipImage (String [] imagePath, int imageCnt)), extract features from images (mipsGetFeature (Bitmap Bitmap)) interface, such as mipsFaceVipDB, mipsFaceVipDB

- *void mipsSetPicPitchAngle(float threhold)/public float mipsGetPicPitchAngle()*

Definition：Set/Get Image detection face pitch angle threshold (greater than this Angle will not be reported as a face)

API table：

Parameters/return value	Type	Description	Example
Return value/ threhold	float	Detection of face pitch angle threshold (suggested setting range :<15.0)	

Notice：This interface set parameters affect the add faces library (addOneFaceToDB (Context Context, mipsFaceVipDB vipFace)), photographs, check (verifyVipImage (String [] imagePath, int imageCnt)), extract features from images (mipsGetFeature (Bitmap Bitmap)) interface, such as mipsFaceVipDB, mipsFaceVipDB

- *void mipsSetVIPDetectFaceWidthThrehold(int width)*

Definition：Set the minimum pixel of VIP (1: N) face frame width for real-time detection during tracking [default: 90] (i.e. face size reaches 90 pixels before VIP recognition).

API table：

Parameters/return value	Type	Description	Example
width	int	The minimum pixel width of a face frame	

Notice： Pay attention to distinguish the interface and mipsSetFaceWidthThrehold, mipsSetFaceWidthThrehold Settings is to track the testing face size threshold, and the interface Settings is to track the detection of the VIP face size threshold, in principle, the threshold value should be greater than or equal to track detection face when threshold (face threshold can be set smaller tracking detection, in the distant place can detect human face, which can make face box, etc., but for the VIP identification accuracy, The VIP face size threshold should be larger than the tracking face size threshold. For example, if the tracking face size threshold is 45, the distance between the corresponding face and the camera is 2.5m. mipsSetFaceWidthThrehold detection VIP face size threshold is set to 90, the corresponding face distance from the camera is 1.7 meters, when someone appears at 2.5 meters can identify someone in the picture, but only within 1.7 meters will identify whether this person is a VIP in the face library.

- *int mipsGetVIPDetectFaceWidthThrehold( )*

Definition：Obtain the minimum pixel of VIP face frame width detection in real time during tracking.

API table：

Parameters/return value	**Type**	**Description**	**Example**
Return value	Int	The minimum pixel width of a face frame	

- *void mipsEnableFaceMoveDetect()/void mipsDisableFaceMoveDetect()*

Definition：Enable/disable face movement detection (off by default). when *flgFaceMotionless* is 1 in the *mipsFaceInfoTrack*, it means if face motion is within the set time（set using interface *mipsSetFaceMoveRitio_THRESHOLD*）, it will keep in that period(set using interface *mipsSetFaceMoveDetectTimer*). Then it be treated as static. If the function is enabled, only when the face is detected as static face, then start the searching face function, feature extract function, liveness detection function. It will increase the first detection accuracy, but it will increase the recognition time when face is moving.

- *int mipsGetFaceMoveDetectState ()*

Definition：Get the current state of face movement detection function.

API table：

Parameters/return value	**Type**	**Descripti on**	**Example**
Return value	Int	1: enable 0: disabled	

- *void mipsSetFaceMoveRitio_THRESHOLD (float ritio)*

Definition：Set the face movement detection threshold (default: 10).Only when the movement amount of the face is less than the set threshold and the duration is continuously set can the face be regarded as the static state;

API table：

Parameters/return value	**Type**	**Description**	**Example**
ritio	float	Movement detection threshold	

- *float mipsGetFaceMoveRitio_THRESHOLD()*

Definition：Get the face movement detection threshold (default: 10).Only when the amount of movement of the face is less than the set threshold and the duration is continuously set can the face be regarded as static state；

API table：

Parameters/return value	**Type**	**Description**	**Example**
Return value	float	Movement detection threshold	

- *void mipsSetFaceMoveDetectTimer(int timer)*

Definition：Set the duration of face movement detection (default: 2 (frame number)).

API table：

Parameters/return value	Type	Description	Example
timer	int	Duration of movement detection	

- *int mipsGetFaceMoveDetectTimer()*

Definition：Get the duration of face movement detection (default: 2 (frame number of screen)).

API table：

Parameters/return value	Type	Description	Example
Return value	int	Duration of movement detection	

- *void mipsSetFaceMoveDetectRatioMode(int mod)*

Definition：Set face movement detection mode (default: 1).

API table：

Parameters/return value	Type	Description	Example
mod	int	0: Ratio mode (face moving distance/face frame width) 1: Absolute value mode (absolute value of face moving pixels)	

Notice：For example: in this interface is set to 1 (absolute value model), *mipsSetFaceMoveRitio_THRESHOLD* set to: 20.0, *mipsSetFaceMoveDetectTimer* set length is 5, in the face of each frame relative to no more than 20 pixels frame to move on, and lasts for 5 frame, can be regarded as stationary.

- *int mipsGetFaceMoveDetectRatioMode()*

Definition：Get the face movement detection mode (default: 1).

API table：

Parameters/return value	Type	Description	Example
Return value	int	0: Ratio mode (face moving distance/face frame width) 1: Absolute value mode (absolute value of face moving pixels)	

- *void mipsEnableIDFeatureCropFace()/void mipsDisableIDFeatureCropFace()*

Definition：Enable/disable face capture feature extraction (off by default).When enabling this function, mBitmapFace of mFaceInfoTrack is effective, and when extracting features from mipsGetFeature, mBitmapFace is also effective. Through mipsSetIDProCropFaceBorderwidth mipsFaceInfoTrack mfaceFeature intercept the border around the image width.

- *int mipsGetIDFeatureCropFaceState ()*

Definition：Get the current status of the face screenshot function.
API table：

Parameters/return value	**Type**	**Description**	**Example**
Return value	Int	1：enable 0：disable	

- *void mipsSetIDProCropFaceBorderwidth(int width)/ int mipsGetIDProCropFaceBorderwidth()*

Definition：Set/Get Screenshot border width (default: 0). The width (pixels) from the edge of the face frame to the edge of the screenshot

- *void mipsSetLivenessMode(int mod)*

Definition：Liveness detection mode is set;(default: 1), that is, fast search only supports dual-camera

API table：

Parameters/return value	**Type**	**Description**	**Example**
mod	int	1. Fast in liveness recognition with dual-camera support only 0: Non-fast in liveness recognition	

Notice： In fast mode can avoid electronic screen, in non-fast mode can avoid electronic screen and most paper photos

- *int mipsGetLivenessMode()*

Definition：Get liveness detection mode.

- *void mipsEnableFaceVerifyArea()/void mipsDisableFaceVerifyArea()*

Definition：Enable/disable VIP and liveness recognition areas (off by default).When this function is enabled, the scope of VIP and liveness detection area can be set through *mipsSetFaceVerifyArea*, that is, VIP recognition and liveness detection will only be carried out when the face is in the set area.

- *int mipsGetFaceVerifyAreaState()*

Definition：Get the functional status of VIP and liveness detection area state.

API table：

Parameters/return value	**Type**	**Description**	**Example**
Return value	Int	1：enable 0：disable	

- *void mipsSetFaceVerifyArea(Rect rect)/ Rect mipsGetFaceVerifyArea()*

Definition：Set/Get VIP and live identification area (pixel).

- *void mipsSetMaxVipVerifyCntThrehold(int cnt)/ int mipsGetVipVerifyCntThrehold()*

Definition：Set/Get VIP on the refresh function is used to identify the VIP number threshold, the VIP on the refresh if camera images of human faces have been identify adults will face the repository has been testing, until reach the threshold number of Settings, pay attention to the threshold is set too big (the default is 2), otherwise the system resource consumption is larger, and may lead to deterrent to hidden trouble.

- *void mipsSetVIPRollAngle(float threhold)/ float mipsGetVIPRollAngle()*

Definition：Set/Get roll angle threshold of "VIP face database search" face (range 1~89, recommended 5~45, default: 20).

Notice： Note the difference from mipsSetRollAngle interface. MipsSetRollAngle sets the tracking roll angle threshold, while mipsSetVIPRollAngle sets the VIP search roll angle threshold, which should be less than the tracking roll angle threshold in theory (such as the tracking roll angle threshold of 30 and the VIP roll angle threshold of 20).

- *void mipsSetVIPYawAngle(float threhold)/ float mipsGetVIPYawAngle()*

Definition：Set/Get yaw angle threshold of "VIP Face Database search" face (range 1~89, recommended 5~45, default: 25)

Notice： Note the difference between mipsSetYawAngle and mipsSetYawAngle interface, mipsSetVIPYawAngle set is the tracking yaw angle threshold, while mipsSetVIPYawAngle set is the VIP search yaw angle threshold, which should be less than the tracking level corner threshold in theory (such as the tracking yaw angle threshold of 30, VIP yaw angle threshold of 25).

- *void mipsSetVIPPitchAngle(float threhold)/ float mipsGetVIPPitchAngle()*

Definition：Set/Get Pitch threshold of "VIP Face Database search" face (range 1~89, recommended 5~45, default: 20)

Notice： Note the difference from mipsSetPitchAngle interface. MipsSetPitchAngle sets the tracking pitch threshold, while mipsSetVIPPitchAngle sets the VIP search pitch threshold, which should be less than the tracking pitch threshold theoretically (e.g., tracking pitch threshold 30, VIP pitch threshold is 20).

2.2.4 Face comparison and face database

- *int initFaceDB(Context context ,String dbFacePath, String imageFacePath , int flgSaveToDisk)*

Definition：Initializes the face database

API table：

Parameters/return value	**Type**	**Description**	**Example**
context	Context		
dbFacePath	String	The face feature library holds the full path (note: The following path is recommended: context.getFilesdir (). GetAbsolutePath (), Context.getfilesdir (). GetAbsolutePath ()+ file.separator +"mipsVipFaceDB". *MipsVipFaceDB* is the file name of the face feature database, not a folder.)	
imageFacePath	String	Add the original image save path of the face library (not in use, standby)	
flgSaveToDisk	int	Whether the face feature database is saved in flash memory 0: don't save 1: save	
Return value	int	0: Face database not found, re-create; 1: Find the existing face database;	

Notice： context.getFilesDir().getAbsolutePath() is recommended for face storage.

- *int saveFaceDB(String dbFacePath)*

Definition：Save the current face database to a file

API table：

Parameters/return value	**Type**	**Description**	**Example**
dbFacePath	String	Save path of face database	
Return value	int	0: saved； -1：failed；	

Notice：V1 and V2 algorithm add or delete face database operation, can only change the memory of the face database, only call this interface to save to the file;V3 version of the algorithm added and deleted after the completion of the operation has been updated to the face database file;

- *boolean mipsCheckVipIdValid(int vipId)*

Definition：Detect the validity of the specified vipId (is it already in the face database)

API table：

Parameters/return value	Type	Description	Example
vipId	int	VipId to be tested	
Return value	boolean	True: The vipId does not exist in the face database; False: The vipId already exists in the face database;	

- *int addOneFaceToDB(Context context ,mipsFaceVipDB vipFace)*

Definition：Add a face (photo) to the face database

API table：

Parameters/return value	Type	Description	Example
context	Context		
vipFace	mipsFaceVipDB	For the face information to be added, the caller should at least set the imagePath (that is, the photo storage path) and ID in the VIP face library. If added successfully, this function will set idxInFaceDB	
Return value	int	0: Successful <0: Add failure: -1: Failed to get the picture bitmap; -2: Abnormal SDK feature extractor; -3: Face ID parameter (Mipsfacevipdb.idxinfacedb) set less than 0 or id duplicate -4: Image conversion fails; -5: The resolution of the image exceeds the requirement (4096*2688); -6: SDK face detector is abnormal. -7: No face detected. -8: Face detection fails. -9: Operation of adding face fails. -10: More than the maximum number of faces. -11: Face size is smaller than preset pixel threshold. -12: Face score is too low. -13: The face roll angle exceeds the threshold. -14: Face yaw angle exceeds the threshold. -15: Face pitch angle exceeds the threshold;	When adding a face fails, the Return value is -11~ -15 VerifyVipImage retrieves human faces Detailed information, and then re-collect to meet the Find the face image or through the following connection Mouth to adjust (relax, such a disadvantage Is the error rate that can lead to VIP identification Will increase) corresponding threshold: -11: Face pixel threshold setting mipsSetPicFaceWidthThrehold -12: Setting the score threshold mipsSetVerifyScoreThrehold -13: Face roll threshold setting mipsSetPicRollAngle -14: Set the yaw threshold of face mipsSetPicYawAngle -15: Face pitch threshold setting mipsSetPicPitchAngle

Notice：The faces added to the face library should be guaranteed to have a positive face, a face score greater than 0.7, a yaw Angle, a pitch Angle and a roll Angle less than 10, and a face frame width greater than 90 pixels

- *int addOneFaceToDB(Context context ,byte[] feature, int idxInFaceDB)*

Definition：Add a face (feature data) to the face database

API table：

Parameters/return value	**Type**	**Description**	**Example**
context	Context		
feature	byte[]	If the face feature vector to be added is added successfully, this function will set idxInFaceDB	
idxInFaceDB	int	**ID in VIP Face database (to be set by user)**	
Return value	int	0: Successful <0: Add failure: -1: Feature is empty; -2: ABNORMAL SDK feature extractor; IdxInFaceDB < 0 or id After); -9: Operation of adding face fails; -10: More than the maximum number of faces;	

Notice：Cooperate with *mipsGetFeature* interface;

- *int deleteOneFaceFrDB(Context context, int idxInFaceDB)*

Definition： Removes the specified face feature from the database

API table：

Parameters/return value	**Type**	**Description**	**Example**
context	Context		
idxInFaceDB	int	Face feature ID (in the Face Feature database)	
Return value	int	0: Delete successfully; -1: Failed to delete	

- *int deleteAllFaceFrDB(Context context)*

Definition：Delete all face features in the face database

API table：

Parameters/return value	**Type**	**Description**	**Example**
context	Context		
Return value	int	0: Delete successfully; -1: Failed to delete	

- *int verifyVipImage(String[] imagePath, int imageCnt)*

Definition： Check and input VIP photo quality, return the serial number with the best photo quality (counted from 0)

API table：

Parameters/return value	**Type**	**Description**	**Example**
imagePath	String[]	The path of the photo to be verified	
imageCnt	int	Number of photos to be verified	
Return value	int	>=0: the serial number with the best quality; -1: Failed to verify, no photos available	

- *mipsFaceVerifyInfo verifyVipImage(Bitmap sourceBitmapFace)*

Definition： Check input VIP picture quality, return photo check details (mipsFaceVerifyInfo. InfoList), detailed information including: image resolution, photographs, all face face width (pixels), pitching Angle, roll Angle, yaw Angle, face score value (is below the set threshold is not considered a human face) and threshold value of these details.

API table：

Parameters/return value	**Type**	**Description**	**Example**
sourceBitmapFace	Bitmap	Bitmap of photos to be verified	
Return value	mipsFaceVerifyInfo	Photo face verification information	

- *float mipsVerifyID(mipsFaceFeature featureImage,mipsFaceFeature featureID)*

Definition： Verify whether two face features (featureImage, featureID) are the same person;

API table：

Parameters/return value	**Type**	**Description**	**Example**
featureImage	mipsFaceFeature	Get facial features from the camera	
featureID	mipsFaceFeature	Features of faces in pictures	
Return value	float	The similarity of two face features, if the value is -1, then the feature in the parameter is null	

Notice： ccording to the application scenario, similarity threshold (API: SimilarityThrehold) should be set. The higher the similarity threshold is, the lower the false detection rate will be, but the slower the detection speed will be. Users should set this threshold according to their needs.

Example code：

```
mipsFaceFeature featureID = mfaceDetect.mipsGetFeature(sourceBitmapFace);
mipsFaceFeature featureImage = mfaceDetect.mipsGetFeature(tmp,PREVIEW_WIDTH,PREVIEW_HEIGHT); ret =
mfaceDetect.mipsVerifyID(featureImage,featureID); if(ret > 0.8) {
        //process code for match okay
}else{
   //process code for match failed
}
```

- *int mipsVerifyInFaceDB(mipsFaceFeature feature)*

Definition： Check whether the camera screen is in the face library;

API table：

Parameters/return value	**Type**	**Description**	**Example**
feature	mipsFaceFeature		Extracted facial features
Return value	Int	>=0: ID in the face database; -3: Verification is completed. It is not in the face database -1: Feature is empty -2: Feature. MfaceFeature feature is null	Match result

Notice： According to the application scenario, similarity threshold (API: SimilarityThrehold) should be set. The higher the similarity threshold is, the lower the false detection rate will be, but the detection rate will be reduced. Users should set this threshold according to their needs.

- *int mipsVerifyInFaceDB(byte[ ] frameImage , int frame_width , int frame_height)*

Definition： Check whether the camera screen is in the face database;

Parameters/return value	**Type**	**Description**	**Example**
frameImage	byte[]	MJPEG data of an image obtained from an RGB camera	
frame_width	int	Width of RGB image (pixels)	
frame_height	int	Width of RGB image (pixels)	
Return value	int	The comparison results >=0: ID in the face database; -3: Verification is completed. It is not in the face database -1: frameImage is empty -2: Face detection in camera screen failed	

Notice： The API will use the largest face frame in the frameImage as the comparison object, and the face frame must be larger than the minimum face frame (this threshold can be set through the API interface: mipsSetFaceWidthThrehold);

- *mipsFaceFeature mipsGetFeature(Bitmap bitmap)*

Definition：Get the feature vector of the face;

API table：

Parameters/return value	**Type**	**Description**	**Example**
bitmap	Bitmap	Face pictures with features to be extracted	
Return value	mipsFaceFeature	Face feature vector & Face frame position	

Notice： Can be used with addOneFaceToDB(Byte [] feature)

Example code：

- *mipsFaceFeature mipsGetFeature(Bitmap bitmap,boolean isIDCard)*

Definition：Get the feature vector of the face (suitable for the id card and other pictures with poor face quality), and force the extraction without checking the face quality of the picture;

API table：

Parameters/return value	**Type**	**Description**	**Example**
bitmap	Bitmap	Face pictures with features to be extracted	
boolean	isIDCard	Picture of ID card: True: yes False: no	
Return value	mipsFaceFeature	Face feature vector & Face frame position	

Notice： Can be used with addOneFaceToDB(Byte [] feature)

- *mipsFaceFeature mipsGetFeature(byte[] frameImage, int frame_width, int frame_height)*

Definition：Get the feature vector of the face;

API table：

Parameters/return value	**Type**	**Description**	**Example**
frameImage	byte[]	Camera input screen of feature to be extracted (NV21 format)	
frame_width	int	The width of image	
frame_height	int	The height of image	
Return value	mipsFaceFeature	Face feature vector & Face frame position	

Notice： Can be used with addOneFaceToDB(Byte [] feature)

- *mipsFaceFeature mipsGetFeature(byte[] frameImage, int frame_width, int frame_height， int flgTackOnly)*

Definition：Get the feature vector of the face;

Parameters/return value	**Type**	**Description**	**Example**
frameImage	byte[]	Camera input screen of feature to be extracted (NV21 format)	
frame_width	int	The width of image	
frame_height	int	The height of image	
flgTackOnly	int	When flgTackOnly is 0, face tracking is only performed, and features are not calculated	
Return value	mipsFaceFeature	Face feature vector & Face frame position	

- *mipsFaceFeature mipsGetFeatureFrDB(int id)*

Definition：Get the face feature with specified ID from the face feature database;

API table：

Parameters/return value	**Type**	**Description**	**Example**
id	int	The ID specified by the user for the face feature database	
Return value	mipsFaceFeature	Face feature vector & Face frame position	

- *List<mipsDeleteResult> deleteBulkFacesFrDB(Context context , int[] ids)*

Definition：Bulk deleting face features from the database

API table：

Parameters/return value	**Type**	**Description**	**Example**
context	Context		
ids	int[]	Face feature ID (in face feature database) array	
Return value	List<mipsDeleteResult>	Return Value for all collections of ids that failed to delete, MipsDeleteResult contains the ID that failed to delete And the corresponding delete failure error code Return after all elements in IDS have been removed successfully The set of returns has length 0.Note: Error code -1 means ID does not exist, -2 means the number of deletions According to the library error	

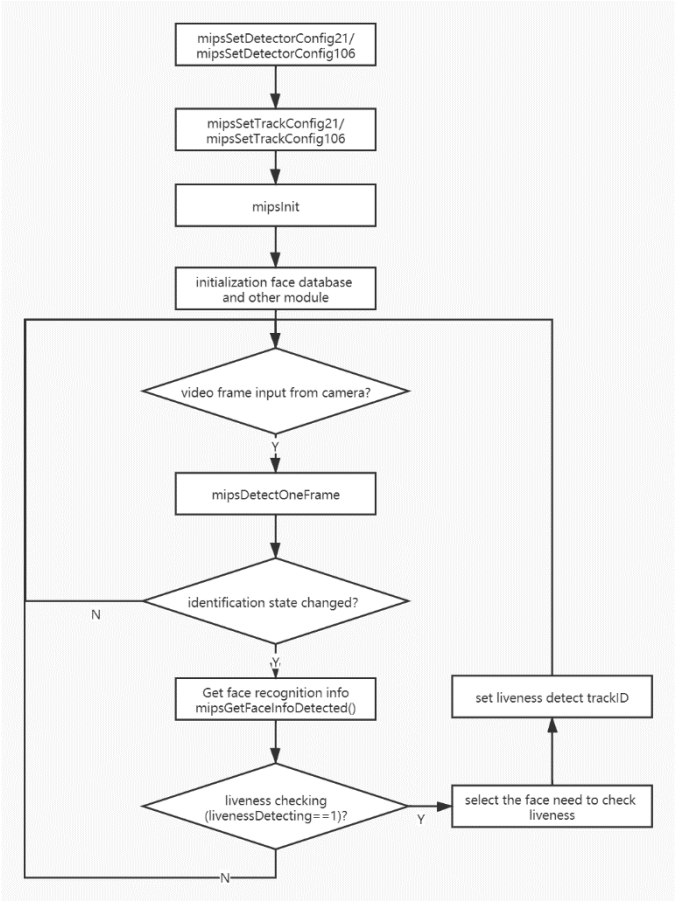
III. module application

In the demo, the usage method of mipsVideoFaceTrack class is in the MipsCameraService class, note:

1、MipsDetectOneFrame function should be mutex set with all parameters, that is, not to be executed at the same time, otherwise it may cause errors in the underlying databaase;

2、 During the initial batch face library creation, it is better to disable the face library detection function (call mipsDisableVipFaceVerify), and then open VIP face library recognition (call MipsenablevipeveriFY) after initialization.This step is not required in subsequent sheet additions.

3、 API recommended usage process:



4、 Typical applications

（1）initialization

SDK initialization includes authorization initialization (), parameter setting (mipsSetDetectorConfig21, mipsSetDetectorConfig21, mipsSetTrackConfig21, mipsSetTrackConfig21, mipsSetTrackConfig106), face database initialization (initFaceDB), reference code：

```
mfaceDetect = new mipsVideoFaceTrack();
int ret=mfaceDetect.mipsInit(context, 2, 90 , “/sdcard/mipsLic/mipsAi.lic”,  0);
if(ret < 0)
{
        return -1;
}
mfaceDetect.mipsSetSimilarityThrehold(0.7f);
mfaceDetect.mipsSetFaceWidthThrehold(90);
mfaceDetect.mipsEnableRefreshFaceRect();
VIP_DB_PATH=context.getFilesDir().getAbsolutePath()+File.separator;
mfaceDetect.initFaceDB(context,VIP_DB_PATH+"mipsVipFaceDB",VIP_DB_PATH+"image",1);
mfaceDetect.mipsEnableVipFaceVerify();
```

（2）Real-time recognition (including face database search, live recognition, face feature vector extraction from camera)

Real-time identification is to point to: input the camera picture of real-time identification SDK (mipsDetectOneFrame), identify the end, through the interface for identifying information (mipsGetFaceInfoDetected), information including (mipsFaceInfoTrack) : face tracking ID (FaceTRrackID), face repository location (faceRect), live test (test results: flgLiveness, face feature vector:MfaceFeature) and facial database search results (ID: FaceIdxDB and similarity: mfaceSimilarity). Among them, mfaceFeature generated after in vivo detection can be used for human identity comparison, and enable/disable related functional blocks via interface:

mipsEnableVipFaceVerify/mipsDisableVipFaceVerify (enabled/VIP is prohibitedFace recognition),
mipsEnableIDFeature/mipsDisableIDFeature (enabled/ban human face feature extraction [living testing success]),
mipsEnableRefreshFaceLiveness/mipsDisableRefreshFaceLiveness (enabled/ban live test refresh),
mipsEnableRefreshFaceVIP/mipsDisableRefreshFaceVIP (enabled/banVIP information refresh)

reference code：

```
lockFaceDb.lock();

flgFaceChange = mfaceTrackLiveness.mipsDetectOneFrame(tmp, PREVIEW_WIDTH,
PREVIEW_HEIGHT,mtrackLivenessID);

lockFaceDb.unlock();

if (flgFaceChange == 1) {


   mcntCurFace = mfaceTrackLiveness.mipsGetFaceCnt(); mFaceInfoDetected =
   mfaceTrackLiveness.mipsGetFaceInfoDetected();
   mtrackLivenessID=getLivenessTrackID(mFaceInfoDetected); //get face to liveness check
   if (mOverlayCamera != null) {

      mOverlayCamera.addFacesLiveness(mFaceInfoDetected, FaceCanvasView.ANALYSIS_STATE);
      mOverlayCamera.postInvalidate();


   }
}
```

（3）Add face database

There are two ways to add face features: directly pass photos to SDK and add face photos to face feature library (*addOneFaceToDB(Context Context,mipsFaceVipDB vipFace));*In another way, *mipsGetFeature* can be extracted first. Apk can save the obtained array of feature vectors, and then add the face features to the face feature database (*addOneFaceToDB(Context Context, Byte [] feature, int idxInFaceDB)* when adding the face features to the face database.

（4）1:1 recognition（non-liveness）

Liveness face recognition is not required, the face 1:1 recognition (compare the photo with the face in the camera). *MipsGetFeature (Bitmap Bitmap)* can be used to obtain the face features in the photo (or the face features in the pre-extracted photo can be used).Then, *mipsGetFeature(Byte [] frameImage, int frame_width, int frame_height,int flgTrackOnly)* API is used to extract face features in the camera screen. To compare the two face feature similarity (*mipsVerifyID (mipsFaceFeature featureImage, mipsFaceFeature featureID)*), you can get the comparison results.

Reference code：pls refer to the API demo *com.example.smdt.smdtaisdkdemo. IDCompareFaceProActivity*

5、MIPS_AI_Basic_BinocularIDCheckOffline.jar support single camera devices and dual-camera devices. And it supports liveness detection for both types of devices.
For dual-camera device, the API in this document all supported. Call *mipsInit*() method and set *flgLiveness* as 2. Using the dual camera interfaces for real-time recognition.
For single camera device, the *flgLiveness* in *mipsInit*() should be set as 0 or 1.

IV. Common interface definition

com.smdt.facesdk.mipsFaceInfoTrack

public int flgInUse; //SDK is in use

public int flgSetLiveness; // flag for liveness detection (only when flgSetLiveness is 1, flgLiveness is valid value)

public int flgSetIDPro; //real-time face feature extract complete (only when flgSetLiveness is 1，mfaceFeature is valid value)

public int flgSetVIP; // VIP identify complete (only when flgSetVIP is 1，FaceIdxDB is valid value)

public int FaceTRrackID; //continuous frame detection, face track ID can be used as face ID.

public int flgLiveness; // result of liveness detection : 1：liveness 0：not passed；
public int livenessDetecting; // status of liveness detection：1：checking, need to input image to SDK. 0：liveness not in use.
public int FaceIdxDB; // VIP face ID in face database
public float mfaceSimilarity; // similarity value of face recognition

public float mfaceScore; //face recognition score

public mipsFaceFeature mfaceFeature; //real-time face feature (if *flgSetLiveness* is 1，*mfaceFeature* is a valid value)
public Rect faceRect; // location of face detected rectangular frame

public int flgLivenessChanged; // liveness detection status

public int flgVerifyChanged; //VIP face database searching status value

public int flgIDProChanged; // 1:1 identify the real-time face feature status change

public mipsFaceFeature mfaceFeature; // face feature data used for searching face database

public int flgFaceMotionless; // flag value state face motion status (1: static 0: motion)

public long vipTimeDebug; // time cost of searching face (ms, for debug)

public long LivenessTimeDebug; //time cost of liveness detection (ms, for debug)

public long allTimeDebug; //debug

public Bitmap mBitmapFaceIR ： // IR image
public Bitmap mBitmapRGB : // image captured from RGB camera

public float mYaw; // yaw angle

public float mPitch; // pitch angle

public float mRoll; // roll angle

public String name ; // name that stored in database

public int isMask; //Check if the mask is wore

com.smdt.facesdk.**mipsFaceVipDB**

public String imagePath; //path of storing VIP face image.

public int idxInFaceDB; // VIP ID stored in face database. (-1, not VIP)

public int vipID; //VIP ID stored in server.

com.smdt.facesdk.**mipsFaceFeature**

public byte[] mfaceFeature; //face feature data
public Rect mfaceRect; // location of face detected rectangular frame
public float mScore //match score of face recognition
public int ID //trackID
public float mYaw //face checked yaw angle
public float mPitch // face checked pitch angle
public float mRoll // face checked roll angle
public Bitmap mBitmapFace //crop face image(include the face frame, width of frame is defined by *mipsSetIDProCropFaceBorderwidth*)

com.smdt.facesdk.**mipsFaceVerifyInfo**

public List<mipsFaceDetailedInfo> infoList; // all the face info, each list for each face. Include face angle, size, score, coordinates, resolution and check result.
public int resultCode; // Return value of checking the image, it is the image check value if there is one face detected. If not multiple face detected, the value will be the largest face that detected. 0 for passing the check, false will return -1~-15
public int faceCnt; // face number of current image
public float PIC_SCORE_THRESHOLD; // VIP add face identify threshold value, default 0.7.
public float PIC_YAW_THRESHOLD; // VIP add face yaw angle threshold value, default 15°.
public float PIC_PITCH_THRESHOLD;// VIP add face pitch angle threshold value, default 15°.
public float PIC_ROLL_THRESHOLD; // VIP add face roll angle threshold value, default 15°.
public float PIC_FACE_WIDTH_THRESHOLD; // VIP add face width threshold value, default 90.
public int PIC_RESOLUTION_X_THRESHOLD; // VIP add image resolution width fixed value, 4096.
public int PIC_RESOLUTION_Y_THRESHOLD; // VIP add image resolution width fixed value, 2688.

com.smdt.facesdk.**mipsFaceDetailedInfo**

public float mScore; //score
public float mYaw;// yaw angle
public float mPitch; // pitch angle
public float mRoll; //roll angle
public float mWidth; // width
public int mResolutionX; // image resolution x
public int mResolutionY; // image resolution y
public Rect mfaceRect; // face detect rectangular frame
public int ret; // current face check value. If only one face detected the value is the same as *mipsFaceVerifyInfo.resultCode.* the value in mipsFaceVerifyInfo.resultCode is the largest face detected.

com.smdt.facesdk.**mipsDeleteResult**

public int userID; /** user id */
public int result; /** delete faild code; -1, userID not exist; -2, delete error */